Group Assignment: Reasoning About Mock-Based Tests

192-211 Automated Software Testing · Siam University · Groups of 3-5

Objective

You are given eight pairs of production code and unit tests. Every test already passes. Your task is not to describe the code. Your task is to explain **why the test is built the way it is**, and to produce evidence for each explanation by changing something and observing what happens.

For each example you will:

1. Predict the result of an experiment before you run it.
2. Run the experiment and paste the real terminal output.
3. Explain the mechanism that produced that output.
4. Explain what the test would and would not catch in a real project.

An answer that only restates what the code does, or only defines a term such as "a mock is a fake object", scores nothing. An answer is only complete when it refers to output your group produced.

Part 0: Setup (required before any question)

Work on one Linux shell or one virtual environment per group.

```
mkdir mock_lab && cd mock_lab
python3 -m venv .venv
source .venv/bin/activate
pip install pytest
```

Create one file per module listed in the examples below. Use exactly these file names, because two of the examples patch by module name and will behave differently if the file is named something else.

File	Contains
`weather.py`	`WeatherService`
`weather_api.py`	`WeatherApi` (Example 1 experiment)
`login.py`	`LoginService`
`verification.py`	`VerificationService`
`idgen.py`	`IdGeneratorUser`
`filesvc.py`	`FileService`
`tax.py`	`TaxService`
`date_utils.py`	`current_year`, `report_name`
`greetings.py`	`get_name`, `greeting`
`report.py`	Example 7 experiment
`test_*.py`	one test file per example

Each test file must import the code under test, for example `from weather import WeatherService`.

Q0.1 Run `pytest -q` and paste the full summary line. State how many tests passed and how long the whole suite took.

Q0.2 Nothing in this suite reads a file, opens a socket, or sleeps. Using your timing from Q0.1, estimate how long these eight tests would take if each one contacted its real dependency over a network. State the assumption behind your estimate.

Example 1: `return_value`

Production Code — `weather.py`

```
class WeatherService:
    def __init__(self, api):
        self.api = api

    def current_temperature(self):
        return self.api.get_temperature()
```

Test — `test_weather.py`

```
from unittest.mock import Mock
from weather import WeatherService

def test_current_temperature():
    api = Mock()
    api.get_temperature.return_value = 25
    service = WeatherService(api)
    assert service.current_temperature() == 25
```

Reasoning Questions

Q1.1 State the behaviour this test verifies in one sentence, then state one behaviour a reader might assume it verifies but which it does not.

Q1.2 The number 25 appears twice, once in the setup and once in the assertion. Explain what is actually being compared, and explain why a test that asserts a value it supplied itself is still worth writing here.

Q1.3 Suppose the test called the real weather API. List three distinct ways that test could fail on a day when `WeatherService` contains no defect at all. For each one, say whether a developer looking at the red test would be able to tell it was not their fault.

Q1.4 `Mock()` creates attributes on demand. Explain, from the pasted output of Experiment 1A, what this means for a test that contains a spelling mistake.

Experiment 1A — predict, then run

Write your prediction first. Then add this file and run it.

```
# exp_spec.py
from unittest.mock import Mock
from weather import WeatherService
from weather_api import WeatherApi          # see weather_api.py below

def test_typo_plain_mock():
```

```
    api = Mock()
    api.get_temparature.return_value = 25   # deliberate typo
    service = WeatherService(api)
    assert service.current_temperature() == 25

def test_typo_with_spec():
    api = Mock(spec=WeatherApi)
    api.get_temparature.return_value = 25   # the same typo
    service = WeatherService(api)
    assert service.current_temperature() == 25
```

```
# weather_api.py
class WeatherApi:
    def get_temperature(self):
        raise NotImplementedError("talks to the real service")
```

Run `pytest -q exp_spec.py --tb=line` and paste both failure lines.

Q1.5 The two tests fail for different reasons. Quote the two error messages and explain which of the two tells a developer where the real mistake is. Say which form of mock you would require in your own project, and name one cost of that choice.

Example 2: Verifying a Method Call

Production Code — `login.py`

```
class LoginService:
    def __init__(self, logger):
        self.logger = logger

    def login(self, username):
        self.logger.info(f"{username} logged in")
```

Test — `test_login.py`

```
from unittest.mock import Mock
from login import LoginService

def test_login_logs_message():
    logger = Mock()
    service = LoginService(logger)
    service.login("Alice")
    logger.info.assert_called_once_with("Alice logged in")
```

Reasoning Questions

Q2.1 `login()` returns nothing. Explain what is left to assert on when a method has no return value, and why that forces a different style of test from Example 1.

Q2.2 `assert_called_once_with` checks three things at once. Name all three. Then describe a defect in `login()` that each one would catch and the other two would miss.

Q2.3 Logging is often called a side concern. Argue either that asserting on a log call is valuable, or that it is over-specification that will break on the next harmless wording change. Commit to one position and give a rule your group would follow.

Experiment 2A — predict, then run

```
# exp_login_variants.py
from unittest.mock import Mock
from login import LoginService

def test_wrong_text():
    logger = Mock()
    LoginService(logger).login("Alice")
    logger.info.assert_called_once_with("alice logged in")

def test_misspelled_assertion():
    logger = Mock()
    LoginService(logger).login("Alice")
    logger.info.assert_called_once_wiht("Bob logged in")
```

Run `pytest -q exp_login_variants.py --tb=line` and paste both results.

Q2.4 `test_misspelled_assertion` names a method that does not exist and asserts about the wrong user. Explain from the output what Python did with it, and whether the protection you observed would still apply to a misspelling such as `logger.inf.assert_called_once_with(...)`. Run that variant if you are not sure, and paste the result.

Q2.5 Replace the mock with a real `logging.Logger` writing to a file. Describe the steps the test would then need in order to make the same assertion. Count the lines and name the part most likely to fail on another machine.

Example 3: Dependency Injection

Production Code — `verification.py`

```
class VerificationService:
    def __init__(self, sms_gateway):
        self.sms_gateway = sms_gateway

    def send_code(self, phone):
        return self.sms_gateway.send(phone, "1234")
```

Test — `test_verification.py`

```
from unittest.mock import Mock
from verification import VerificationService

def test_send_code():
    gateway = Mock()
    gateway.send.return_value = "SENT"
    service = VerificationService(gateway)
    assert service.send_code("0812345678") == "SENT"
```

Reasoning Questions

Q3.1 Name the injected dependency and the exact line where the substitution becomes possible.

Q3.2 Rewrite `__init__` so that the gateway is constructed inside the class instead of being passed in. Then try to write a test for `send_code` against that version without touching the production code. Report how far you got and where you stopped.

Q3.3 This test asserts only on the returned string. Add `gateway.send.assert_called_once_with("0812345678", "1234")` and run it. Paste the result, then state what this second assertion reveals about `send_code` that the first one hides.

Q3.4 The verification code `"1234"` is fixed in the production code. Explain whether this is a defect, and explain why the existing test passes either way. State the smallest change to the test that would make the choice visible to a

reviewer.

Experiment 3A — predict, then run

Make `gateway.send` return `"FAILED"` instead of `"SENT"` and run the original test unchanged.

Q3.5 Paste the output. The production code has not changed. Explain what this tells you about where the asserted value comes from, and what that means for how much confidence a passing run of this test should give you.

Example 4: `side_effect` as a List

Production Code — `idgen.py`

```
class IdGeneratorUser:
    def __init__(self, generator):
        self.generator = generator

    def create_three_ids(self):
        return [
            self.generator.next_id(),
            self.generator.next_id(),
            self.generator.next_id(),
        ]
```

Test — `test_idgen.py`

```
from unittest.mock import Mock
from idgen import IdGeneratorUser

def test_multiple_ids():
    generator = Mock()
    generator.next_id.side_effect = [1, 2, 3]
    service = IdGeneratorUser(generator)
    assert service.create_three_ids() == [1, 2, 3]
```

Reasoning Questions

Q4.1 Replace `side_effect = [1, 2, 3]` with `return_value = 1` and run the test. Paste the failure. Explain from that output what property of the production code the list form is checking and the single form cannot.

Q4.2 Name two real dependencies that return a different value on each call, and one that is expected to return the same value every time. Say which of `side_effect` and `return_value` fits each, and why.

Q4.3 The assertion compares a list, so order matters. Describe a defect in `create_three_ids` that produces the right three values in the wrong order, and say whether this test would catch it.

Experiment 4A — predict, then run

```
# exp_fourth_id.py
from unittest.mock import Mock

def test_fourth_call():
    g = Mock()
    g.next_id.side_effect = [1, 2, 3]
    g.next_id(); g.next_id(); g.next_id()
    g.next_id()
```

Run `pytest -q exp_fourth_id.py --tb=line`, paste the error name, and then run it again with a fourth element added to the list.

Q4.4 Name the exception raised on the fourth call. Explain why this is useful feedback rather than a nuisance, and describe the change in production code that would cause a real test suite to start failing this way.

Example 5: `side_effect` as an Exception

Production Code — `filesvc.py`

```
class FileService:
    def __init__(self, storage):
        self.storage = storage

    def save(self, data):
        try:
            self.storage.write(data)
            return "OK"
        except IOError:
            return "FAILED"
```

Test — `test_filesvc.py`

```
from unittest.mock import Mock
from filesvc import FileService

def test_disk_failure():
    storage = Mock()
    storage.write.side_effect = IOError("Disk full")
    service = FileService(storage)
    assert service.save("data") == "FAILED"
```

Reasoning Questions

Q5.1 Describe, as a procedure another student could follow, how you would make a real disk return "no space left" on demand. Then estimate the time and the risk of doing that on the machine you are working on now.

Q5.2 This test proves the failure path returns `"FAILED"`. Name one thing the function should arguably also do when a write fails, and explain why this test passing says nothing about it.

Q5.3 The `except` clause names `IOError`. Explain what happens to a storage backend that raises a different exception type, using the output from Experiment 5A as evidence.

Experiment 5A — predict, then run

```
# exp_wrong_exception.py
from unittest.mock import Mock
from filesvc import FileService

def test_other_error():
    storage = Mock()
    storage.write.side_effect = ValueError("bad data")
    service = FileService(storage)
    assert service.save("data") == "FAILED"
```

Run it and paste the result. Note whether pytest reports a failed assertion or something else.

Q5.4 State the difference between a test that fails on its assertion and a test that fails because an exception escaped the code under test. Explain which one you observed here and how you could tell from the output alone.

Q5.5 Both a happy path and a failure path now exist for `save`. Write the missing happy-path test, run it, and paste the result. Explain why the failure test on its own was not enough evidence that `save` works.

Example 6: `side_effect` as a Function

Production Code — `tax.py`

```
class TaxService:
    def __init__(self, calculator):
        self.calculator = calculator

    def total(self, amount):
        return self.calculator.calculate(amount)
```

Test — `test_tax.py`

```
from unittest.mock import Mock
from tax import TaxService

def add_tax(amount):
    return amount * 1.07

def test_tax_calculation():
    calculator = Mock()
    calculator.calculate.side_effect = add_tax
    service = TaxService(calculator)
    assert service.total(100) == 107
```

Reasoning Questions

Q6.1 Explain what the mock receives, what it passes on, and what it returns. Then state what breaks if `total` forgets to pass `amount` through. Verify your claim by making that change and pasting the output.

Q6.2 A function was used here instead of a fixed value or a list. State the condition under which each of the three forms is the right choice. Give the rule in terms of the dependency's behaviour, not in terms of syntax.

Q6.3 The mock now contains arithmetic. Argue whether `add_tax` has become a second implementation that must itself be kept correct, and say what you would do if the real tax rule changed.

Experiment 6A — predict, then run

```
# exp_tax_five.py
from unittest.mock import Mock
from tax import TaxService

def add_tax(amount):
    return amount * 1.07

def test_tax_five():
    calculator = Mock()
    calculator.calculate.side_effect = add_tax
    service = TaxService(calculator)
    assert service.total(5) == 5.35
```

Predict pass or fail, then run `pytest -q exp_tax_five.py` and paste the assertion line in full.

Q6.4 The original test asserts `== 107` and passes. This one asserts `== 5.35` and does not. Explain the difference using the exact numbers pytest printed. State the change you would make to the assertion, and say whether that change hides a real defect or removes a false alarm.

Example 7: `patch` as a Context Manager

Production Code — `date_utils.py`

```
def current_year():
    return 2026

def report_name():
    return f"Report-{current_year()}"
```

Test — `test_date_utils.py`

```
from unittest.mock import patch
from date_utils import report_name

def test_report_name():
    with patch("date_utils.current_year") as mock_year:
        mock_year.return_value = 2030
        assert report_name() == "Report-2030"
```

Reasoning Questions

Q7.1 In Examples 1 to 6 the dependency arrived through `__init__`. Here it does not. Explain what `report_name` depends on and why that dependency cannot be injected without editing the production code.

Q7.2 Write the file below, run `pytest -q -s exp_print_scope.py`, and paste both printed lines. Explain what the context manager did on entry and on exit.

```
# exp_print_scope.py
import date_utils
from unittest.mock import patch

def test_scope():
    with patch("date_utils.current_year") as mock_year:
        mock_year.return_value = 2030
        print("inside:", date_utils.current_year(), date_utils.report_name())
    print("outside:", date_utils.current_year(), date_utils.report_name())
```

Q7.3 State what would happen to the other tests in the suite if the replacement were not undone at the end of the block. Say whether the order in which pytest runs tests would then matter.

Experiment 7A — predict, then run

Create a second module that imports the function by name.

```
# report.py
from date_utils import current_year

def report_name():
    return f"Report-{current_year()}"
```

```
# exp_patch_target.py
from unittest.mock import patch
from report import report_name

def test_patch_wrong_target():
    with patch("date_utils.current_year") as mock_year:
        mock_year.return_value = 2030
        assert report_name() == "Report-2030"

def test_patch_right_target():
    with patch("report.current_year") as mock_year:
        mock_year.return_value = 2030
        assert report_name() == "Report-2030"
```

Run `pytest -q exp_patch_target.py --tb=line` and paste both outcomes.

Q7.4 One passes and one fails, and the two patch strings differ by one word. Explain what each string names, using the value printed in the failure message as your evidence. Then state the rule your group will use to choose a patch target, in a form that works for any module.

Q7.5 Explain why the original `test_date_utils.py` passes with `"date_utils.current_year"` even though `exp_patch_target.py` does not. Point to the difference in how each module obtained the name.

Example 8: `patch` as a Decorator

Production Code — `greetings.py`

```
def get_name():
    return "Real User"

def greeting():
    return f"Hello {get_name()}"
```

Test — `test_greetings.py`

```
from unittest.mock import patch
from greetings import greeting

@patch("greetings.get_name")
def test_greeting(mock_name):
    mock_name.return_value = "Alice"
    assert greeting() == "Hello Alice"
```

Reasoning Questions

Q8.1 Name what is replaced and for how long. Compare the lifetime of the replacement here with the lifetime in Example 7, and say which is easier to get wrong when a test file grows to twenty tests.

Q8.2 The test function takes a parameter `mock_name` that nothing in the test file calls it with. Explain where that argument comes from. Then delete the parameter, run the test, and paste the error.

Q8.3 Remove the decorator but keep the assertion, run it, and paste the failure. Explain what the message `Hello Real User` versus `Hello Alice` tells you about what patching actually changed.

Experiment 8A — predict, then run

Patch two names on the same test.

```
# exp_two_patches.py
from unittest.mock import patch
from greetings import greeting
from date_utils import report_name

@patch("greetings.get_name")
@patch("date_utils.current_year")
def test_order(a, b):
    a.return_value = 2030
    b.return_value = "Alice"
    assert greeting() == "Hello Alice"
    assert report_name() == "Report-2030"

@patch("greetings.get_name")
@patch("date_utils.current_year")
def test_order_swapped(a, b):
    a.return_value = "Alice"
    b.return_value = 2030
    assert greeting() == "Hello Alice"
```

Predict which parameter receives which mock, then run `pytest -q exp_two_patches.py --tb=line` and paste both results.

Q8.4 State the order in which stacked decorators bind to parameters, with the output as evidence. Say what a group should do to make this unmistakable to the next reader of the test.

Q8.5 Both the context manager and the decorator form replace the same function. Give one situation where you would choose each, in terms of the scope of the test rather than personal taste.

Part 9: Synthesis

Answer these after all eight examples. Each answer must cite at least one example or output from above.

Q9.1 Every test in this assignment passes, and in every case the value being asserted was chosen by the test author. Explain what this class of test can establish about a program, and what it cannot. Be specific about which kind of defect survives a fully green mock-based suite.

Q9.2 Examples 1 to 6 receive their dependency as an argument. Examples 7 and 8 reach into a module to replace a name. State which of the two you would prefer in new code and why. Then state what you would do when the code is not yours to change.

Q9.3 For each of the eight examples, decide whether mocking was necessary, merely convenient, or arguably wrong. Present your verdicts in a table with a one-line reason each. A verdict of "mocking is not justified here" is a correct answer if you can defend it.

Q9.4 Pick the one experiment above whose result surprised your group most, and say what you now check for in your own tests because of it. If no result surprised you, say so and give the prediction you were most confident about and why.

Submission Instructions

- Work in groups of 3 to 5. One submission per group.
- Write roughly 200 to 300 words of explanation per example, plus the pasted output. Pasted output does not count towards the word total.
- Paste output as text. Screenshots are not accepted.
- Answer every numbered question. Keep the question numbers.
- List which member ran which experiment at the top of the report.
- Combine everything into one report and convert it to PDF.

- Name the file `MockReasoning_Group<NN>.pdf`.

Your working `mock_lab` directory must still contain every file named above, including the `exp_*.py` experiment files, in the state that produced your pasted output.

Marking

Each numbered question is marked in one of three bands.

Band	Criteria
Full	The answer explains the mechanism in the group's own words and is supported by output the group produced. A prediction that turned out wrong is still full marks when the group explains why.
Partial	The reasoning is correct but generic. It would apply to any mock, cites no output, or answers only part of what was asked.
None	The answer restates a definition, describes the code line by line, contradicts the group's own pasted output, or pastes output with no explanation.

Two further conditions apply to the whole report.

- Output that cannot be reproduced from the submitted files is treated as absent, and the questions depending on it drop to the None band.
- A report that reaches the right conclusions but records no predictions is capped at the Partial band, because the predictions are the part being assessed.